Application development for Android

Constraint Layout

Custom Views

View Model

Constraint Layout

- Support for complex layouts: Constraint Layout offers robust support for creating large and intricate UI layouts
- Similar to Relative Layout: It shares similarities with the Relative Layout, particularly regarding the establishment of relationships and dependencies with other views
- Key Features Include:
  - Bias: Allows precise control of view alignment within constraints
  - Dimension Relations: Control size and spacing relationships
  - Aspect Ratio: Support for maintaining view proportions
  - Guidelines: Assist in precise positioning and alignment
  - Chains: Simplify the management and alignment of interconnected views
  - Barriers: Facilitate dynamic and responsive layouts

  … among others

Custom Views

- New *Views* can be defined by deriving new classes from the `View` or one of its subclasses

- The behavior can be programmed overriding specific methods related to events that occur on the view

- Override the `'on*'` methods:
  - `onDraw`: This method is called whenever the view needs to be redrawn. It can be triggered programmatically by calling the `'invalidate'` method of the custom view.
  - `onKeyDown`
  - `onTouchEvent`
  - `onSizeChanged`
  - …

Custom view

- A custom view must have constructors compatible with the `View` class

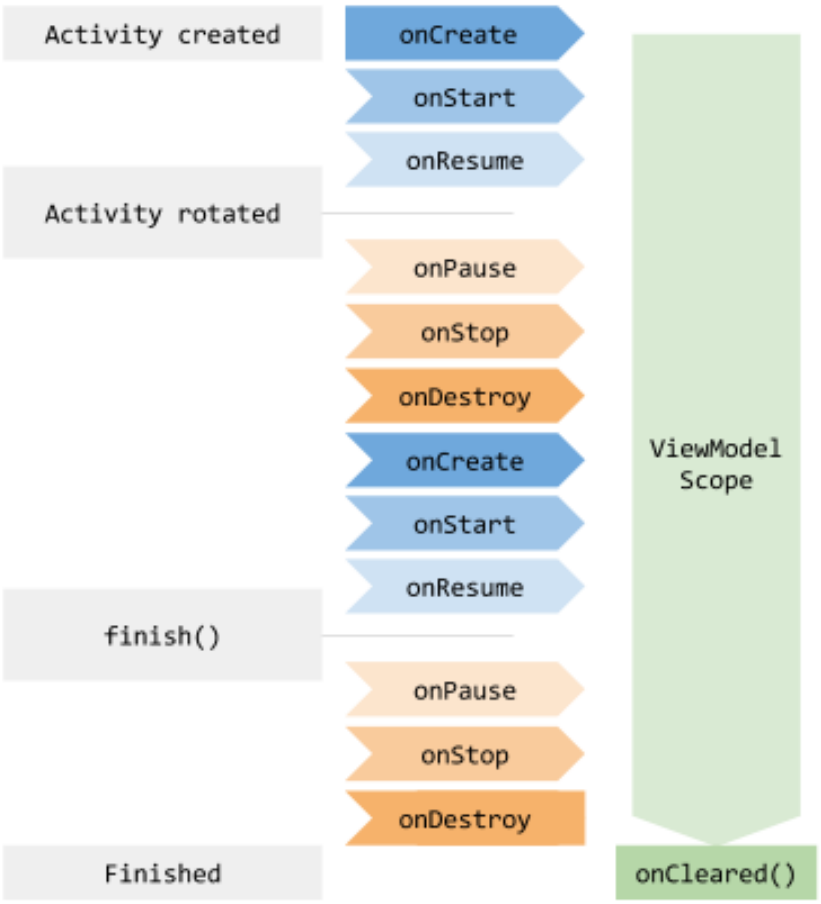
```
class TicTacToeView : View {
    constructor(context : Context) : super(context)
    constructor(context : Context,
                attrs : AttributeSet?) :
                                    super(context,attrs)
    constructor(context : Context,
                attrs : AttributeSet?,
                defStyleAttr:Int) :
                      super(context,attrs,defStyleAttr)

}
```

  - Default parameters can be used to avoid multiple definitions of the constructor (but lose some compatibility with *Java*)
  - If the custom view is only intended for programmatic use, it can have only the primary constructor with the `Context`

ViewModel

- The `ViewModel` class allows to implement the *view-model* component of the MVVM *software design pattern*, already studied in *Advanced Programming* subject
- It can be used to temporarily store data of a related component (e. g., an activity)
  - Can be redefined to support implementation of an activity's entire data model
  - The object is not deleted with unexpected changes to the environment, such as screen rotations
  - Must not reference any *View*



ViewModel

- Creating a *View Model* object is done by extending a new class from the `ViewModel` class

```
class MyViewModel : ViewModel() { ... }
```

- The reference to an object of this type is obtained by delegating this functionality, managed internally, to a variable of the created type
  - In `build.gradle.kts (module)`:

```
implementation("androidx.fragment:fragment-ktx:1.8.4")
```

  - In the activity where the view model object will be used:

```
val model : MyViewModel by viewModels()
```

    - In the context of a *fragment*, if it is needed to access the activity's `ViewModel`, then "`by activityViewModels()`" should be used

ViewModel

```
class MyViewModel : ViewModel() {
    private var _value = 0
    val value : Int
        get() = _value++
}

class MainActivity : AppCompatActivity() {

    val viewModel : MyViewModel by viewModels()

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        Log.i("Test", "onCreate: ${viewModel.value}")
    }

    override fun onResume() {
        super.onResume()
        Log.i("Test", "onResume: ${viewModel.value}")
    }

    override fun onPause() {
        super.onPause()
        Log.i("Test", "onPause: ${viewModel.value}")
    }
}
```

LiveData

- The `LiveData` objects allow to implement the *Observer software design pattern* taking into account the lifecycle of Android components
  - Provides methods for registering *Observers*
  - Only notifies active observers about updates
    - Active observers are those in the STARTED or RESUMED states
  - It has protected methods to change the respective value: `postValue` and `setValue`

- `MutableLiveData`
  - `LiveData` extension that exposes the `postValue` and `setValue` methods, changing them to `public`

ViewModel and MutableLiveData

```
class MyViewModel : ViewModel() {
    private var _value = 0
    val value : Int
        get() = _value++

    private val _state : MutableLiveData<String> by lazy { MutableLiveData<String>("none") }
    val state : LiveData<String>
        get() = _state

    fun changeState(newState: String) { Log.i("Test", "changeState: $newState $_value ${this.hashCode()}")
        _state.setValue("$newState $value")
    }
}
class MainActivity : AppCompatActivity() {
    val viewModel : MyViewModel by viewModels()

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        viewModel.state.observe(this) {  Log.i("Test", it)  }
        viewModel.changeState("onCreate ${this.hashCode()}")
    }
    override fun onResume() {
        super.onResume()
        viewModel.changeState("onResume ${this.hashCode()}")
    }
    override fun onPause() {
        super.onPause()
        viewModel.changeState( "onPause ${this.hashCode()}")
    }
    override fun onStop() {
        super.onStop()
        viewModel.changeState( "onStop ${this.hashCode()}")
        //viewModel.state.removeObservers(this)
    }
}
```

```
changeState: onCreate 251924405 0 147911348
onCreate 251924405 0
changeState: onResume 251924405 1 147911348
onResume 251924405 1
changeState: onPause 251924405 2 147911348
onPause 251924405 2
changeState: onStop 251924405 3 147911348
changeState: onCreate 242710700 4 147911348
onCreate 242710700 4
changeState: onResume 242710700 5 147911348
onResume 242710700 5
changeState: onPause 242710700 6 147911348
onPause 242710700 6
changeState: onStop 242710700 7 147911348
changeState: onCreate 91623694 8 147911348
onCreate 91623694 8
changeState: onResume 91623694 9 147911348
onResume 91623694 9
```

Exercise

- Tic Tac Toe
  - A data model already available at the following *gist*:
    - https://gist.github.com/ansisec
  - The bridge between the activity and the data model should be established through a `ViewModel`
  - The Tic Tac Toe board should be implemented using a single *Custom View*
  - The *Custom View* should be programmatically added to a `FrameLayout` included in the activity layout
    - The activity layout should be designed using a `ConstraintLayout` as its base

